PYTHON FOR DATA SCIENCE · MODULE 1 · TOPIC 1.22

EDA Mini Project

Day 21 — a real dataset, from raw file to written findings



Boston University

2	4	6
Projects: one taught, one yours	Data files provided	Steps in the workflow

Prepared & presented by **Srinivasa Sai Chava**

WHY THIS MATTERS

Everything from the last six sessions, in one piece of work

Day 15	read_csv, inspect, derive columns
Day 16	filter rows, loc, reset_index
Day 17	groupby, agg, transform
Day 18	merge two tables, check row counts
Day 19	missing values, sentinels, duplicates
Day 20	outliers, matplotlib, seaborn

The difference between knowing the tools and being able to use them

Every session so far gave you a clean example and told you which method to use. Today nobody tells you. You get two files and five questions, and the work is deciding what to do — which is the only part that is actually difficult. Expect to spend more time looking at the data than writing code.

THE METHOD

The six-step EDA workflow

1. **Load & look**
   shape, head, dtypes, info — before anything else
2. **Audit**
   missing, sentinels, duplicates, odd values
3. **Clean**
   normalise text, then deduplicate, then decide on gaps
4. **Combine**
   merge the lookup table, check the row count
5. **Explore**
   groupby and plots — answer the questions
6. **Write up**
   findings, in sentences, with the caveats

The order matters. Auditing before cleaning stops you fixing things that were not broken. Normalising text before deduplicating is the Day 19 rule. Merging before cleaning multiplies your mess. And writing up last means your findings describe the data you actually ended with, not the one you started with.

TODAY'S WORK

Two projects — I do one, you do the other

PROJECT A

Retail orders

I walk through this one end to end, on screen, thinking out loud — including the wrong turns.

```
orders.csv       423 rows  x 7 cols
customers.csv     81 rows  x 5 cols
```

Follow along, but do not race ahead — the point is the reasoning, not the typing.

PROJECT B

Flight delays

Yours. Same six steps, different data, different problems — and nobody tells you which method to reach for.

```
flights.csv      511 rows  x 7 cols
airlines.csv       6 rows  x 3 cols
```

Submit the notebook plus five written findings. Marking is on the next-to-last slide.

⚠️ **Both datasets are deliberately dirty.** Every problem in them is one we have covered: sentinel values pandas will not flag, inconsistent text, duplicate rows, a key that matches nothing, and outliers that are NOT errors. If your first numbers look suspiciously clean, you have not audited properly.

1

Project A — Retail Orders

The walkthrough · 45 minutes

- The data
- Five questions
- Steps 1 to 6 live

Project A — what you are given

orders.csv — 423 rows

Column	Meaning
`order_id`	unique per order — in theory
`customer_id`	links to customers.csv
`order_date`	2025, as text
`category`	product group
`quantity`	units ordered
`unit_price`	price per unit, Rs
`discount_pct`	0 to 20

customers.csv — 81 rows

Column	Meaning
`customer_id`	the key
`name`	customer name
`city`	one of five
`segment`	Consumer / Corporate / Home Office
`signup_year`	2019 to 2025

81 rows for 80 customers. That one extra row is your first finding.

Revenue is not a column — you have to build it

```
revenue = quantity * unit_price * (1 - discount_pct/100)
```

Which means every problem in quantity, unit_price and discount_pct flows straight into your headline number. Clean those three before you compute anything — and decide what a missing discount means before you let it become a missing revenue.

Project A — the five questions

1. How much revenue did we take in 2025, and from how many valid orders?
2. Which city generates the most revenue — and is that just because it has more customers?
3. Which product category sells most by revenue, and which by units?
4. Do Corporate customers behave differently from Consumer ones?
5. Are there any orders we should investigate before trusting these numbers?

Question 2 is the one that separates a report from an analysis. A total is not a comparison — revenue per customer may tell a completely different story.

Steps 1 to 3 — load, audit, clean

```
#  --- 1. LOAD & LOOK
orders = pd.read_csv("orders.csv")
print(orders.shape)
print(orders.dtypes)
orders.head()

#  --- 2. AUDIT  (before touching anything)
orders.isna().sum()                # what pandas sees
orders["unit_price"].describe()    # a min of -999?
orders["category"].value_counts()  # odd entries?
orders.duplicated().sum()
orders["order_id"].duplicated().sum()

#  --- 3. CLEAN  (in this order)
orders["category"] = (orders["category"]
                      .str.strip().str.title())
orders = orders.replace(-999, np.nan)
orders = orders.drop_duplicates()
orders["discount_pct"] = orders["discount_pct"].fillna(0)
```

What the audit will tell you

- 34 missing discounts — visible to isna()
- 11 unit prices of -999 — NOT visible
- 7 categories of “unknown” — NOT visible
- 17 distinct categories, for 4 real ones
- 2 exact duplicate rows
- 3 repeated order_ids
- 1 customer_id in orders that is in no customer record

Only the first of those seven is found by isna(). The rest need describe(), value_counts() and duplicated() — which is why step 2 exists as a step.

Why fillna(0) for the discount? Because “no discount recorded” almost certainly means no discount was given — and here the business confirms it. State that assumption in your write-up.

Steps 4 and 5 — combine, then explore

```
#  --- 4. COMBINE
cust = cust.drop_duplicates(subset="customer_id")

before = len(orders)
df = orders.merge(cust, on="customer_id",
                  how="left", validate="many_to_one")
assert len(df) == before
print("orders with no customer:",
      df["city"].isna().sum())

#  --- 5. EXPLORE
df["revenue"] = (df["quantity"] * df["unit_price"]
                 * (1 - df["discount_pct"] / 100))

by_city = df.groupby("city").agg(
    revenue   = ("revenue", "sum"),
    orders    = ("order_id", "size"),
    customers = ("customer_id", "nunique"),
)
by_city["rev_per_customer"] = (by_city["revenue"]
                               / by_city["customers"])
```

⚠️ Deduplicate the LOOKUP first

customers.csv has 81 rows for 80 customers. Merging without removing that duplicate silently duplicates every order belonging to them — Day 18's row explosion, in a real file. The assert catches it.

```
validate="many_to_one"  would also raise
```

Answering question 2 properly

The city with the highest total revenue may simply have the most customers. Dividing by the customer count turns a total into a comparison — and the ranking can change completely. That one extra line is the difference between reporting and analysing.

Step 6 — the part most people skip

⚠️ Not a finding

```
city
Mumbai      2,841,330
Pune        2,104,880
Delhi       1,998,210
```

A table is evidence. On its own it makes the reader do the work of deciding what it means — and they will usually decide wrong, or not bother.

A finding

“Mumbai produced the most revenue overall, but it also has the most customers. Per customer, Delhi is ahead — so Mumbai's lead is a matter of size, not of performance. (Based on 418 valid orders; 5 were excluded as duplicates or unmatched.)”

A claim, a comparison, and the caveat.

Every finding needs three things

WHAT	SO WHAT	CAVEAT
the number or the pattern	why it matters, or what it contradicts	what you cleaned, dropped or assumed

2

Project B — Flight Delays

Your turn · the rest of the session

- The data
- Five questions
- What to hand in

Project B — what you are given

flights.csv — 511 rows

Column	Meaning
`flight_id`	unique per flight — in theory
`airline_code`	links to airlines.csv
`origin`	departure airport
`destination`	arrival airport
`distance_km`	route distance
`passengers`	number on board
`delay_min`	minutes late; 0 means on time

airlines.csv — 6 rows

Column	Meaning
`airline_code`	the key
`airline_name`	full name
`founded`	year founded

Six airlines listed. Check how many actually appear in flights.csv — and whether every code in flights.csv appears here.

⚠️ Three things to be careful about

A delay of 0 is ON TIME, not missing — do not fill or drop it. · A delay of -999 is a recording failure, not an early arrival. · A passenger count of 0 is not a real flight with nobody aboard. Decide what each of those three means BEFORE you compute an average delay, because all three change it.

Project B — the five questions

1. How many flights are usable, and what did you exclude or repair?
2. Which airline is most punctual? Use a measure you can defend.
3. Which route has the worst delays, and is that based on enough flights to mean anything?
4. Does a longer flight tend to be more delayed?
5. Find the extreme delays. Are they errors, or are they real? Justify your answer.

Question 3 has a trap in it. A route with one catastrophically delayed flight will top any "average delay" ranking — so report the count alongside, and say whether you trust it.

What to hand in, and how it is marked

Submit one notebook containing

- The six steps, as labelled sections
- An audit cell showing what you found wrong
- At least four charts, all labelled
- Five written findings — what, so what, caveat
- A short “what I cleaned and why” note

Marking

Area	Marks
Audit: did you find the problems?	25
Cleaning: right choices, stated	25
Analysis: correct and relevant	20
Charts: readable and labelled	15
Findings: written, with caveats	15

Half the marks are for the data work, not the answers

That is deliberate, and it reflects real practice. A beautifully presented average computed over data you never checked is worth nothing. Two students can reach different numbers and both score well, provided each can say what they did and why — and both will beat a student who reports a single confident figure with no audit behind it.

SELF-REVIEW

Before you submit — the eight checks

Check	Why	One line
Did isna() miss anything?	sentinels are invisible to it	`df.describe()`
Is the text normalised?	17 “categories” for 4 real ones	`value_counts()`
Did you clean before deduplicating?	Day 19: the order changes the answer	—
Did the merge change the row count?	a duplicated key multiplies rows	`assert len(df) == before`
Any keys that matched nothing?	an orphan id is a finding	`indicator=True`
Did you LOOK at the outliers?	they may be real	print the rows
Is every chart labelled?	unlabelled is unfinished	`set_xlabel / set_title`
Does each finding have a caveat?	what you cleaned changes the number	—

If you can answer all eight, your analysis is defensible — even where someone else made different choices.

TIMELINE

How the two hours run

0:00	15 min	Brief, the workflow, and the two datasets
0:15	45 min	Project A walkthrough — I drive, you follow
1:00	50 min	Project B — you work, I walk around
1:50	10 min	Two volunteers show one finding each

If you finish Project B early, answer a sixth question of your own invention — that is what the capstone will ask you to do.

THE WHOLE SESSION IN TWELVE LINES

Recap

1. Six steps: load, audit, clean, combine, explore, write up.
2. Audit BEFORE cleaning. You cannot fix what you have not found.
3. isna() finds only some gaps — describe() and value_counts() find the rest.
4. Normalise text, THEN deduplicate. The order changes the answer.
5. Deduplicate the lookup table before you merge it.
6. assert len(after) == len(before) after every merge.
7. A derived column inherits every problem of its inputs.
8. A total is not a comparison — divide by the count.
9. LOOK at outliers before treating them. Some are the finding.
10. Report the count beside every average.
11. Every chart: both axes labelled, and a title.
12. Every finding: what, so what, and the caveat.

To hand in

Project B as one notebook: the six steps as sections, an audit cell, at least four labelled charts, and five written findings with caveats. The eight-point checklist as your final cell.

Next — Module 1 assessment

Then Module 2: Statistics & Probability. Everything you have just done by eye, you will learn to do with a test and a p-value.